Lesson 10: Ray Tracing

Lesson 10: Ray Tracing 1
10.1 Ray Casting 2
10.2 Intersections 2
10.2.1 Geometric Objects Intersections 2
10.3 The Scene Signature 2
10.4 Efficiency 3
10.5 Surface Normal at Intersection Points 3
10.6 Shading 3
Lesson 10 Discussion Session 4

10.1 Ray Casting

Ray tracing

A rendering method that follows rays of light backwards to eventually find what color they are. It's a technique for higher quality graphics.

Rays may be launched recursively when the color of a point on an object depends on incoming light from specific known directions. For example, when a ray hits a reflective object, a recursive ray is launched in the direction the reflected light is coming from.

In Ray Casting for each pixel in the viewport, cast a ray from the eye position through the point (in eye coordinates) that maps to the pixel center. Intersect the ray with all objects in the scene to find the point closest to the eye position, and apply the illumination model at that point.

Used technique to produce 3D Computer Graphics images. This replaces the z-buffer algorithm.

Limitations of Ray Tracing

- ✓ While ray tracing works well for point lights and directional lights, it can't handle
- ✓ area lights. An area light is one that has are.
- ✓ Ray tracing uses shadow rays to tell whether a light source illuminates an object.
- ✓ But a shadow ray is cast in only one direction, and can only hit one point on an area light.
- ✓ Ray tracing is that it doesn't take into account illumination by reflected light.

10.2 Intersections

10.2.1 Geometric Objects Intersections

The intersection of two geometric objects is the set of points (part of the plane, space) they have in common.

In lines, two line segments can only intersect if their y-span have an overlap,
Two line segments can only intersect if their y-spans have an overlap, and they are adjacent in the x-order at that y-coordinate (they are horizontal neighbors)

10.3 The Scene Signature

Larger image structures, such as windows, signs, or doors, offer more hope as they capture shape and texture on a broader scale.

Such structures forms scene signatures, they can be reliably matched under large variations in appearance, thus opening the door to robust localization.

The benefit of using scene signatures instead of low-level point features, which look for corners, edges, or blobs, is that the data association problem becomes less challenging, since scene signatures are very distinctive (e.g., doors, signs, windows,
etc.).

10.4 Efficiency

Because material properties are part of the state, if we change materials for many surfaces, we can affect performance. We can make the code cleaner by defining a material structure and setting all materials during initialization. We can then select a material by a pointer

```
typedef struct materialStruct {
    GLfloat ambient[4];
    GLfloat diffuse[4];
    GLfloat specular[4];
    GLfloat shineness;
} MaterialStruct;
```

10.5 Surface Normal at Intersection Points

A good surface-intersection technique has to balance three conflicting goals:

- ✓ Efficiency
- ✓ Robustness.
- ✓ Accuracy.

10.6 Shading

Shading -Determine a color for each filled pixel

Texture Mapping-Describe shading variation within polygon interiors

Choosing a color for each filled pixel.

- ✓ Each illumination calculation for a ray from the eye point through the view plane provides a radiance sample
- ✓ How do we choose where to place samples?
- ✓ How do we filter samples to reconstruct image?

Steps in OpenGL shading

1. Enable shading and select model
2. Specify normals
3. Specify material properties
4. Specify lights

Lesson 10 Discussion Session

When would you choose a displacement map over a bump map? Which is likely to be more difficult to implement and why? (Think carefully about the flow of operations in the standard computer graphics pipeline.) Why do you not get a realistic appearance when using an environment map to render the appearance of a large flat mirror? What visual artifacts would you expect to see if you tried to do this?